

Consulta rápida - Homework 2

Funções principais

Função	Receita	Custo/atenção
sort	sort(v.begin(),v.end())	$O(n \log n)$; comparador estrito
lower_bound	primeiro $\geq x$	$O(\log n)$ no vetor ordenado
upper_bound	primeiro $> x$	$O(\log n)$ no vetor ordenado
binary_search	existe x?	faixa ordenada, retorna bool
unique + erase	compacta e reduz duplicatas consecutivas	$O(n)$; ordene para deduplicar tudo
accumulate	accumulate(b,e,0LL)	$O(n)$; 0LL determina acumulador
next_permutation	sort; do/processar/while	$O(n)$ por chamada; P permutações
min/max_element	iterador do extremo	$O(n)$; faixa não vazia

Escolha da estratégia

Gatilho	Estratégia	Hipótese e custo
Muitos valores $\leq x$, $\geq x$, em [L,R]	Ordenação + bounds	$O(n \log n + q \log n)$
Par com soma alvo	Sort + duas pontas	$O(n \log n)$; preserve índices
Maior segmento soma $\leq S$	Janela	Valores ≥ 0 e $S \geq 0$; $O(n)$
Soma exata com negativos	Prefixos + frequências	Extra da semana 1; $O(n \log n)$ com map
Intervalos ativos	Eventos ou bounds	$O((n+q) \log(n+q))$; empates importam
Menor valor viável	Busca na resposta	ok monotônico F...T; hi viável
Maior valor viável	Busca na resposta	ok monotônico T...F
Todas as ordens	next_permutation	$O(nP)$, $P \leq n!$
Incluir/excluir n itens	Bitmask	$O(n \cdot 2^n)$; n pequeno
Restrições durante escolhas	Backtracking	Fazer/chamar/desfazer; pior caso exponencial

Fórmulas e fronteiras

- Ocorrências de x: upper_bound(x)-lower_bound(x).
- Valores em [L,R]: upper_bound(R)-lower_bound(L).
- Intervalos [l,r] contendo x: entradas $\leq x$ menos saídas $< x$.
- Intervalos (l,r) contendo x: entradas $< x$ menos saídas $\leq x$; descarte vazios.
- Intervalos [l,r] contendo x: entradas $\leq x$ menos saídas $\leq x$.
- Prefixos 0-based: soma inclusiva [l,r]=pref[r+1]-pref[l].
- Duas partições: diferença=abs(total-2*soma), se as contas couberem.
- Máscara: bit j = 1ULL<<j; n<64 e custo exponencial viável.
- Rainhas: diagonal linha+coluna; outra linha-coluna+n-1.

Busca de mínimo - fragmento C++17

```
// lo <= resposta <= hi, hi viável; limites não negativos.
while (lo < hi) {
    long long mid = lo + (hi - lo) / 2;
    if (ok(mid)) hi = mid;
    else lo = mid + 1;
}
// lo é a primeira resposta viável.
```

Dicas salvadoras e pegadinhas

- Verifique end() antes de *it. Para predecessor, confira it!=begin().
- Faça multiplicações em long long desde o início: 1LL*a*b.
- Janela por soma não funciona em geral com negativos.
- Sort pode destruir a contiguidade original; pareamento é outro problema.
- Sweep: decida se fronteiras são abertas, fechadas ou semiabertas.
- Busca binária: prove ok monotônico e um limite viável; sature contagens.
- Backtracking: caso base, escolha válida, fazer, recursão, desfazer.
- Não pode podar soma>alvo se ainda há valores negativos.
- Map[x] insere chave ausente; find não insere. Multiset.erase(x) remove todas.
- Cada arquivo tem main próprio; compile separadamente com -std=c++17.
- Teste vazio quando permitido, n=1, duplicatas, limites, impossível e overflow.

Origem

STL e prefixos: aula 1, #1 Introdução.pdf. Busca binária, resposta, two pointers, sweep e backtracking: aula 2, -2 Buscas.pdf. Bitmask e compressão de coordenadas também são oficiais da aula 2; técnicas de teste são apoio adicional. Os detalhes e provas estão no guia explicado.